

JAVA 17 OCP · YDS STUDY SERIES

Unit 11 · Exceptions and Localization — Technical Memory Notes

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu not, [ana çift dilli dersin](#) Java 17 kurallarını hızlı tekrar ve sınav kararı biçiminde sıkıştırır. Sorular özgün OCP tarzı çalışma sorularıdır; gerçek sınav sorusu değildir.

1. Exception hierarchy

```
java.lang.Object
├── java.lang.Throwable
│   ├── java.lang.Error                unchecked
│   └── java.lang.Exception
│       ├── java.lang.RuntimeException  unchecked
│       └── diğer Exception türleri      checked
```

- Checked exception (kontrollü istisna), application code tarafından handle veya declare edilmelidir.
- `RuntimeException` ve subclass'ları unchecked'tir.
- `Error` ve subclass'ları unchecked'tir.
- `Throwable`ı doğrudan extend eden, `Error` veya `RuntimeException` olmayan custom type checked kabul edilir.

Memory tip: “Checked = `Exception` kolunda, fakat `RuntimeException` kolunda değil.”

2. Handle or declare rule

Checked exception üretebilen code path için iki seçenek vardır:

- 1 Compatible `catch` block ile handle etmek.
- 2 Method signature'da `throws` ile declare etmek.

```
void read() throws IOException {
    throw new IOException();
}
```

```
void read() {
    try {
        throw new IOException();
    } catch (IOException e) {
        System.out.println(e.getMessage());
    }
}
```

Unchecked exception için handle/declare zorunlu değildir:

```
void fail() {
    throw new IllegalArgumentException();
}
```

throw ve throws

Keyword	Konum	Görev
<code>throw</code>	Method body	Tek bir <code>Throwable</code> object'i fırlatır
<code>throws</code>	Method declaration	Bir veya daha fazla exception type bildirir

```
void work() throws IOException, SQLException {
    throw new IOException();
}
```

Bir method declare ettiği checked exception'ı gerçekten throw etmek zorunda değildir.

3. Common exception class'ları

Runtime exceptions

Type	Tipik neden
<code>ArithmeticException</code>	Integer division by zero
<code>ArrayIndexOutOfBoundsException</code>	Geçersiz array index
<code>ClassCastException</code>	Runtime object için geçersiz cast
<code>NullPointerException</code>	<code>null</code> reference dereference
<code>IllegalArgumentException</code>	Method'a uygunsuz argument
<code>NumberFormatException</code>	String'i sayıya çevirememesi

`NumberFormatException`, `IllegalArgumentException`ın subclass'ıdır.

`ArrayIndexOutOfBoundsException`, `IndexOutOfBoundsException`ın subclass'ıdır.

Checked exceptions

Type	Package / ilişki
<code>IOException</code>	<code>java.io</code> ; genel I/O hatası
<code>FileNotFoundException</code>	<code>IOException</code> subclass'ı
<code>ParseException</code>	<code>java.text</code> ; parsing başarısızlığı
<code>SQLException</code>	<code>java.sql</code> ; database erişim hatası

Errors

Type	Tipik neden
<code>ExceptionInInitializerError</code>	Static initializer exception ile biter
<code>StackOverflowError</code>	Çoğunlukla bitmeyen recursion
<code>NoClassDefFoundError</code>	Runtime'da gerekli class definition bulunamaz

Error'ları sıradan recovery akışı için catch etmek iyi tasarım değildir; fakat Java syntax'ı birçok **Error** type'ini catch etmeye izin verir.

4. Exception object ve stack trace

```
try {
    throw new IllegalStateException("broken");
} catch (RuntimeException e) {
    System.out.println(e.getMessage()); // broken
    System.out.println(e.getClass().getSimpleName()); // IllegalStateException
}
```

`printStackTrace()` varsayılan olarak `System.err` yazar. `System.out` ve `System.err` farklı stream'ler olduğundan combined display order her ortamda garanti edilmez.

Java 14+ helpful `NullPointerException` mesajları hangi expression'ın `null` olduğuna ilişkin ayrıntı verebilir; exact wording'e sınav cevabı gibi güvenme.

5. Method çağrıları ve checked exception

```
void caller() {
    risky(); // DOES NOT COMPILE
}

void risky() throws IOException {}
```

`risky()` gerçekten exception throw etmese bile declaration caller'ı bağlar. `caller()` ya `throws IOException` eklemeli ya da exception'ı catch etmelidir.

Compiler yalnız ulaşılabilir checked exception olasılığına göre karar verir.

```
try {
    System.out.println("safe");
} catch (IOException e) { // DOES NOT COMPILE
}
```

Try body `IOException` throw edemez; checked catch unreachable'dır. Unchecked exception catch'i bu nedenle genellikle derlenebilir.

6. Override edilen method ve exception

Override method:

- Daha geniş checked exception declare edemez.
- Aynı checked exception'ı declare edebilir.
- Daha dar checked exception declare edebilir.
- Hiç checked exception declare etmeyebilir.
- İstenen unchecked exception'ı declare edebilir.

```
class Parent {
    void work() throws IOException {}
}

class Child extends Parent {
    @Override
    void work() throws FileNotFoundException {}
}
```

Bu kod derlenir; `FileNotFoundException`, `IOException`dan daha dardır.

```
class Child extends Parent {
    @Override
    void work() throws Exception {} // DOES NOT COMPILE
}
```

Reference type çağrı noktasındaki handle/declare kontrolünü belirler:

```
Parent p = new Child();
p.work(); // IOException'a göre handle/declare gerekir
```

7. Traditional `try` syntax

Geçerli ana biçimler:

```
try {
    work();
} catch (Exception e) {
    recover();
}
```

```
try {
    work();
} finally {
    cleanup();
}
```

```
try {
    work();
} catch (Exception e) {
    recover();
} finally {
    cleanup();
}
```

Traditional `try` en az bir `catch` veya `finally` ister. Braces zorunludur.

8. Catch order

Catch block'lar yukarıdan aşağıya kontrol edilir. Daha dar subclass önce gelmelidir:

```
try {
    work();
} catch (FileNotFoundException e) {
    System.out.println("file");
} catch (IOException e) {
    System.out.println("io");
}
```

Tersi **does not compile**, çünkü subclass catch unreachable olur:

```
try {
    work();
} catch (IOException e) {
} catch (FileNotFoundException e) { // DOES NOT COMPILE
}
```

Sibling exception type'ların sırası önemli değildir.

9. *finally* flow

finally, normal completion veya caught/uncaught exception sonrasında genellikle çalışır.

```
static int value() {
    try {
        return 1;
    } finally {
        System.out.print("F");
    }
}
```

value() önce **F** yazdırır, sonra **1** döndürür.

finally içinde **return** yazmak try/catch return'ünü veya pending exception'ı bastırabilir; yasal olsa da kötü pratiktir.

System.exit() gibi JVM'i sonlandıran durumlarda *finally* çalışmayabilir.

Flow çözüm sırası

- 1 *try* içindeki satırları exception'a kadar yürüt.
- 2 İlk compatible *catch* block'u seç.
- 3 *finally* varsa çalıştır.
- 4 Sonra pending return veya exception'ın devam edip etmediğine karar ver.

10. Multi-catch

```
try {
    work();
} catch (IOException | SQLException e) {
    System.out.println(e.getMessage());
}
```

Kurallar:

- Alternatifler | ile ayrılır.
- Tek variable declaration en sonda bulunur.
- Alternatif type'lar birbirinin subclass/superclass'ı olamaz.
- Multi-catch variable implicitly final'dır; yeniden assign edilemez.

```
catch (IOException | FileNotFoundException e) { } // DOES NOT COMPILE
```

`FileNotFoundException`, `IOException`'ın subclass'ıdır; ilk type zaten ikincisini kapsar.

11. Custom exceptions

```
class CannotSwimException extends Exception {
    CannotSwimException(String message) {
        super(message);
    }
}
```

Bu checked exception'dır.

```
class BadTankException extends RuntimeException {
    BadTankException(Throwable cause) {
        super(cause);
    }
}
```

Bu unchecked exception'dır.

Constructor otomatik inherit edilmez; istediğin message/cause constructor'ını açıkça yaz.

12. Try-with-resources temeli

Resource type `AutoCloseable` implement etmelidir:

```
try (var in = new FileInputStream("data.txt")) {
    System.out.println(in.read());
}
```

Try-with-resources (TWR):

- `catch` veya `finally` olmadan yazılabilir.
- Resource declaration'ları semicolon ile ayrılır.
- Resource'ları reverse declaration order'da kapatır.
- Resource'ları try body bittikten sonra, `catch/finally`den önce kapatır.

```
try (Resource a = new Resource("A");
    Resource b = new Resource("B")) {
    System.out.print("T");
}
```

Close order: **B**, sonra **A**.

13. Java 9+ existing resource syntax

Java 17'de try header içinde önceden declaration yapılmış final veya effectively final variable kullanılabilir:

```
var input = new FileInputStream("data.txt");
try (input) {
    System.out.println(input.read());
}
```

Variable sonradan reassign ediliyorsa effectively final değildir ve usage **does not compile**:

```
var input = new FileInputStream("data.txt");
try (input) {}
input = null; // input artık effectively final değil; try satırı derlenmez
```

TWR header içinde declaration yapılan variable'ın scope'u try body ile sınırlıdır:

```
try (var input = new FileInputStream("data.txt")) {
    System.out.println(input.read());
}
System.out.println(input); // DOES NOT COMPILE
```

14. `close()` declaration ve checked exception

`AutoCloseable.close()` signature'ı:

```
void close() throws Exception;
```

Implementation daha dar exception veya hiç exception declare etmeyebilir. Resource'ın declared type'ı TWR'ın hangi checked exception'ı handle/declare etmesini belirler.

```
class Door implements AutoCloseable {
    public void close() throws IOException {}
}

void enter() throws IOException {
    try (Door d = new Door()) {}
}
```

Try body boş olsa bile implicit `close()` nedeniyle `IOException` handle/declare edilmelidir.

15. Suppressed exceptions

TWR'da bir exception zaten primary iken `close()` başka exception throw ederse close exception'ı suppressed olur.

```

class JammedDoor implements AutoCloseable {
    public void close() {
        throw new IllegalStateException("close");
    }
}

try (var door = new JammedDoor()) {
    throw new IllegalArgumentException("body");
} catch (Exception e) {
    System.out.println(e.getMessage()); // body
    System.out.println(e.getSuppressed().length); // 1
    System.out.println(e.getSuppressed()[0].getMessage()); // close
}

```

Primary exception ilk throw edilen `body` exception'ıdır. İki resource reverse order'da kapanıp ikisi de fail ederse suppressed array de close sırasını izler.

Traditional `finally` block'ta atılan yeni exception eski exception'ı suppressed list'e otomatik eklemeyiz; eski exception kaybolabilir.

16. Formatting API haritası

```

java.text
├── NumberFormat
│   ├── getNumberInstance()
│   ├── getCurrencyInstance()
│   ├── getPercentInstance()
│   ├── getIntegerInstance()
│   └── getCompactNumberInstance()
├── DecimalFormat
└── MessageFormat

java.time.format
└── DateTimeFormatter

```

- `format()` object/value → localized `String`
- `parse()` localized `String` → number/temporal result

17. NumberFormat

```

Locale us = Locale.US;
Locale germany = Locale.GERMANY;

System.out.println(NumberFormat.getNumberInstance(us).format(1234.5));
// 1,234.5

System.out.println(NumberFormat.getNumberInstance(germany).format(1234.5));
// 1.234,5

```

Currency symbol ve placement locale'a bağlıdır:

```

NumberFormat money = NumberFormat.getCurrencyInstance(Locale.GERMANY);
System.out.println(money.format(2.4)); // 2,40 €

```

Exact whitespace Unicode non-breaking space olabilir; görselde sıradan space gibi görünebilir.

Percent

```
NumberFormat percent = NumberFormat.getPercentInstance(Locale.US);
System.out.println(percent.format(0.25)); // 25%
```

Input `0.25`, yüzde olarak `25%` olur.

Compact numbers

```
var shortFormat = NumberFormat.getCompactNumberInstance(
    Locale.US, NumberFormat.Style.SHORT);
var longFormat = NumberFormat.getCompactNumberInstance(
    Locale.US, NumberFormat.Style.LONG);

System.out.println(shortFormat.format(100_000)); // 100K
System.out.println(longFormat.format(100_000)); // 100 thousand
```

Locale data JVM implementation'ına göre küçük ayrıntılar gösterebilir; soruda verilen locale ve style'a dikkat et.

18. Number parsing

```
NumberFormat format = NumberFormat.getNumberInstance(Locale.US);
Number value = format.parse("1,234.5");
System.out.println(value.doubleValue()); // 1234.5
```

`NumberFormat.parse(String)` checked `ParseException` declare eder.

Parsing her zaman bütün input'u tüketmeyebilir:

```
Number value = NumberFormat.getNumberInstance(Locale.US).parse("12abc");
System.out.println(value); // 12
```

Strict bütün-string validation gerekiyorsa `ParsePosition` veya ek kontrol gerekir.

19. DecimalFormat pattern

Symbol	Anlam
0	Digit yoksa da zero göster
#	Digit yoksa gösterme
.	Decimal separator placeholder
,	Grouping separator placeholder
%	100 ile çarpıp percent göster
'text'	Literal text

```
var format = new DecimalFormat("000.0#");
System.out.println(format.format(12.3)); // 012.3
System.out.println(format.format(12.34)); // 012.34
```

Pattern symbol'ları locale-aware formatter'da locale karşılıklarıyla gösterilebilir.

20. DateTimeFormatter

Predefined formatter:

```
LocalDate date = LocalDate.of(2022, Month.APRIL, 30);
System.out.println(date.format(DateTimeFormatter.ISO_LOCAL_DATE));
// 2022-04-30
```

Formatter temporal object'ta olmayan field isterse kod derlenir fakat runtime'da

`DateTimeException` (çoğunlukla `UnsupportedTemporalTypeException`) oluşur:

```
date.format(DateTimeFormatter.ISO_LOCAL_DATE_TIME); // runtime exception
```

Common pattern letters

Pattern	Alan
<code>Y, YY, YYYY</code>	Year-of-era
<code>M, MM, MMM, MMMM</code>	Month
<code>d, dd</code>	Day-of-month
<code>E,EEEE</code>	Day-of-week text
<code>h</code>	1–12 hour
<code>H</code>	0–23 hour
<code>m</code>	Minute
<code>s</code>	Second
<code>a</code>	AM/PM
<code>Z</code>	Time-zone name

OCP trap: Uppercase `M` month, lowercase `m` minute'tır.

```
var formatter = DateTimeFormatter.ofPattern(
    "dd MMMM yyyy", new Locale("tr", "TR"));
System.out.println(LocalDate.of(2024, 5, 6).format(formatter));
// 06 Mayıs 2024
```

`z` gibi zone field'ı `LocalDateTımede` yoktur; `ZonedDateTime` gerekir. Literal metin single quote ile escape edilir:

```
DateTimeFormatter.ofPattern("hh 'o''clock' a");
```

21. Locale oluşturma

Locale:

- language code: lowercase (**en**, **tr**, **fr**)
- country/region code: uppercase (**US**, **TR**, **CA**)

```
Locale a = new Locale("en");
Locale b = new Locale("en", "US");
Locale c = new Locale.Builder()
    .setLanguage("tr")
    .setRegion("TR")
    .build();
```

Constructor yanlış case'i normalize edebilse de sınavda standard convention'ı kullan.

Country tek başına geçerli dil bilgisi taşımaz:

```
new Locale("", "US")
```

Object oluşturulabilir; fakat normal requested-locale tasarımı için language code beklenir.

22. Default locale ve category

```
Locale.setDefault(Locale.US);
Locale current = Locale.getDefault();
```

İki category vardır:

```
Locale.Category.DISPLAY
Locale.Category.FORMAT
```

- **DISPLAY**: Locale adlarının nasıl gösterileceği gibi UI text.
- **FORMAT**: Number/date/currency formatting default'ları.

```
Locale.setDefault(Locale.Category.FORMAT, Locale.GERMANY);
```

Bir formatter'a explicit locale verilirse default FORMAT category göz ardı edilir:

```
NumberFormat.getCurrencyInstance(Locale.US)
```

23. Resource bundle adlandırma

Base name **Zoo** için örnek files:

```
Zoo.properties
Zoo_en.properties
Zoo_en_US.properties
Zoo_fr.properties
```

```
ResourceBundle bundle =
    ResourceBundle.getBundle("Zoo", new Locale("en", "US"));
String greeting = bundle.getString("hello");
```

Properties file:

```
hello=Hello
open=The zoo is open
```

Key bulunamazsa `MissingResourceException` runtime'da oluşur.

24. Resource bundle selection

Basitleştirilmiş arama:

```
requested locale:
base_language_COUNTRY
base_language
base

uygun requested chain bulunamazsa:
default-language_COUNTRY
default-language
base
```

Örneğin requested `fr_FR`, default `en_US`:

```
Zoo_fr_FR → Zoo_fr → Zoo
gerekirse Zoo_en_US → Zoo_en → Zoo
```

Bir bundle hierarchy seçildikten sonra eksik key için o hierarchy'nin parent'larına gidilir; key ararken başka locale hierarchy'sine atlanmaz.

Class-based ve `.properties` bundle aynı specificity'de bulunursa class-based bundle tercih edilir.

25. Properties

`Properties`, `Hashtable<Object, Object>`ı extend eder; fakat string property API'sini tercih et:

```
var props = new Properties();
props.setProperty("name", "Ada");
System.out.println(props.getProperty("name")); // Ada
System.out.println(props.getProperty("missing", "default")); // default
```

`getProperty()` yalnız String value döndürür. `put()` ile non-String value eklenirse `getProperty()` onu property olarak görmez:

```
props.put("count", 3);
System.out.println(props.get("count")); // 3
System.out.println(props.getProperty("count")); // null
```

26. MessageFormat

```
String pattern = "Hello, {0}. You have {1} messages.";
String result = MessageFormat.format(pattern, "Ada", 3);
System.out.println(result);
// Hello, Ada. You have 3 messages.
```

Placeholder index'i zero-based'dir. Single quote escape karakteridir:

```
MessageFormat.format("It's {0}", "ready") // It's ready
```

Localized pattern resource bundle'dan alınabilir; argument formatting locale'a göre ayarlanabilir.

27. Compile-time, runtime ve output ayrımı

Durum	Sonuç
Unhandled checked exception	Does not compile
Unchecked exception declare edilmemiş	Derlenebilir
Superclass catch subclass catch'ten önce	Does not compile
Related type'lar multi-catch'te birlikte	Does not compile
TWR existing resource effectively final değil	Does not compile
TWR <code>close()</code> checked exception'ı unhandled	Does not compile
TWR body + close exception	Body primary, close suppressed
<code>LocalDate</code> + time field formatter	Runtime exception
Missing resource key	Runtime <code>MissingResourceException</code>
<code>NumberFormat.parse()</code> exception'ı unhandled	Does not compile
Explicit locale'lı formatter	Default FORMAT locale'dan bağımsız

28. OCP çözüm algoritması

Exception sorusu

- 1 Her `throw` ve method call'un exception type'ını bul.
- 2 Checked mı unchecked mı sınıflandır.
- 3 Handle/declare sağlanmış mı?
- 4 Catch order reachable mı?
- 5 Multi-catch type'ları related mı?
- 6 Override declaration parent'tan daha geniş mi?
- 7 TWR resource'ları reverse order'da kapat.
- 8 Primary ve suppressed exception'ları ayır.
- 9 En son `finally` effect'ini uygula.

Localization sorusu

- 1 Explicit locale var mı, default mu?
- 2 DISPLAY mi FORMAT category mi etkili?
- 3 Format type number/currency/percent/date/time hangisi?
- 4 Pattern'da `M/m`, `O/#`, quote farklarını kontrol et.
- 5 Temporal object gerekli field'lara sahip mi?
- 6 Resource bundle requested hierarchy'yi sırala.
- 7 Key fallback'ı yalnız seçilen hierarchy içinde uygula.
- 8 Compile-time / runtime / exact output ayrımını yap.

29. Mini quiz · Özgün OCP tarzı çalışma soruları

- 1 `RuntimeException` checked mı unchecked mı?
- 2 `catch (IOException | FileNotFoundException e)` neden derlenmez?
- 3 TWR'da `A`, sonra `B` declare edilirse close order nedir?
- 4 Try body exception throw ederken `close()` da exception throw ederse hangisi primary olur?
- 5 `LocalDate` object'ini `"HH:mm"` ile formatlamak hangi aşamada hata verir?
- 6 `NumberFormat.getPercentInstance().format(0.4)` yaklaşık hangi anlamı verir?
- 7 Pattern'da `MM` ve `mm` arasındaki fark nedir?
- 8 Requested `fr_CA` için önce hangi bundle candidate'ları denenir?
- 9 `Locale.Category.DISPLAY`, explicit `Locale.GERMANY` verilen currency formatter'ını etkiler mi?
- 10 `Properties.put("x", 5)` sonrasında `getProperty("x")` ne döndürür?
- 11 Override method parent'taki `IOException` yerine `Exception` declare edebilir mi?
- 12 Empty traditional `try {}` tek başına derlenir mi?

Cevaplar ve açıklamalar

- 1 Unchecked.
- 2 `FileNotFoundException`, `IOException`ın subclass'ıdır; related alternatives aynı multi-catch'te kullanılamaz.
- 3 B, sonra A.
- 4 Try body exception'ı primary; close exception'ı suppressed olur.
- 5 Kod derlenir; runtime'da temporal object hour/minute taşımadığı için exception oluşur.
- 6 `%40 / 40%`; exact symbol placement locale'a bağlıdır.
- 7 `MM` month, `mm` minute.
- 8 `base_fr_CA`, sonra `base_fr`, sonra base bundle.
- 9 Hayır. Explicit locale formatter için belirleyicidir.
- 10 `null`; non-String value generic `get()` ile alınabilir.
- 11 Hayır; daha geniş checked exception declare edemez.
- 12 Hayır. Traditional `try`, en az bir `catch` veya `finally` ister.

Son tekrar kartı

```
checked = Exception - RuntimeException
throw object
throws declaration

catch: child before parent
multi-catch: unrelated types

TWR:
AutoCloseable
reverse close
existing resource → final/effectively final
body exception → primary
close exception → suppressed

format:
0 required digit
# optional digit
M month
m minute

bundle:
requested specific → requested language → base
then default chain if no requested hierarchy
selected hierarchy içinde key fallback
```